

(Multi-Agent) Reinforcement Learning: a primer, with applications in economics

MARL Algorithms and Challenges - extra material

Aldo Glielmo

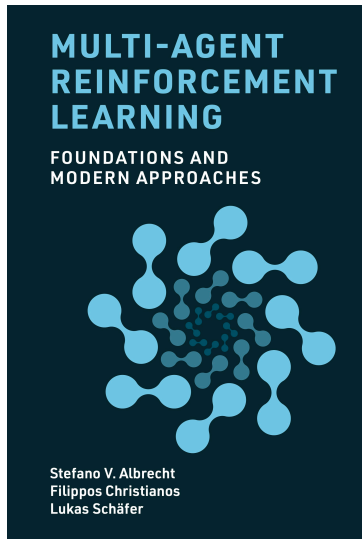
Applied Research Team, Banca d'Italia

A lot of material in this course is based on

Multi-Agent Reinforcement Learning: Foundations and Modern Approaches

The book is freely available from:

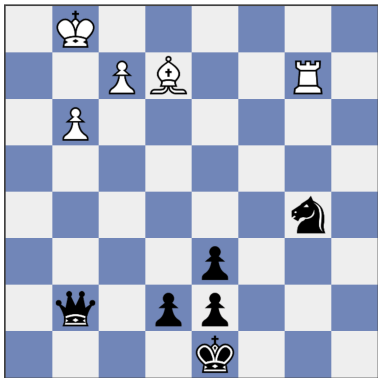
www.marl-book.com



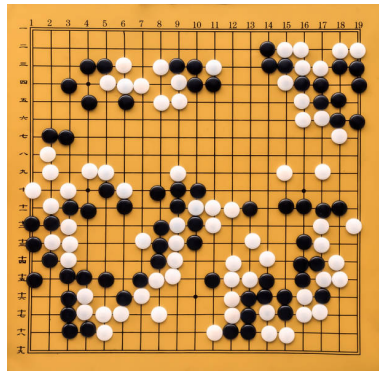
Policy Self-Play in Zero-Sum Games

The Challenge of Zero-Sum Board Games

Next we will take a closer look at (turn-based) zero-sum board games such as chess, shogi, or Go.

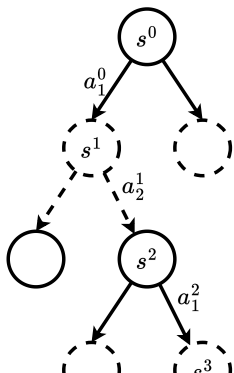


(a) Chess



(b) Go

Tree Search for Zero-Sum Games



Agent 1's turn



Agent 2's turn

We can view turn-based zero-sum games as trees where

- *nodes* represent game states
- *edges* represent actions
- *leaves* represent terminal states

in each node either agent 1 or agent 2 makes a move.

Problem

The tree can grow very large depending on its

- **Depth:** number of

Monte Carlo Tree Search (MCTS)

Monte Carlo Tree Search (MCTS) is a tree-search algorithm that simulates (samples) possible outcomes of the game, and then updating value estimates of actions.

MCTS consists of three key steps that are repeated:

- **Simulation:** simulate a game from the current state until a leaf node is reached
- **Expansion:** expand the tree by adding a new node for the reached state if it does not exist yet
- **Backpropagation:** update the estimated values of the nodes visited during the selection step

Monte Carlo Tree Search – Simulation

MCTS maintains two statistics for each visited state-action pair:

- Value estimates $Q(s, a)$
- Visitation counts $N(s, a)$

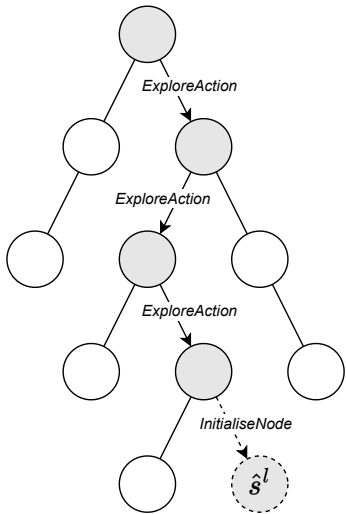
MCTS generates k simulations from the current state by sampling actions according to a policy until a leaf node is reached. Leaf nodes are previously unvisited or terminal states.

To sample actions, MCTS commonly uses ϵ -greedy policies with respect to action-value function Q , or the upper confidence bound (UCB) policy:

$$\hat{a}^\tau = \begin{cases} \hat{a} & \text{if } N(\hat{s}^\tau, \hat{a}) = 0 \\ \arg \max_{\hat{a} \in A} \left(Q(\hat{s}^\tau, \hat{a}) + \sqrt{\frac{2 \ln N(\hat{s}^\tau)}{N(\hat{s}^\tau, \hat{a})}} \right) & \text{otherwise} \end{cases}$$

that incentivises exploration of states and actions that have not been visited often.

Monte Carlo Tree Search – Expansion

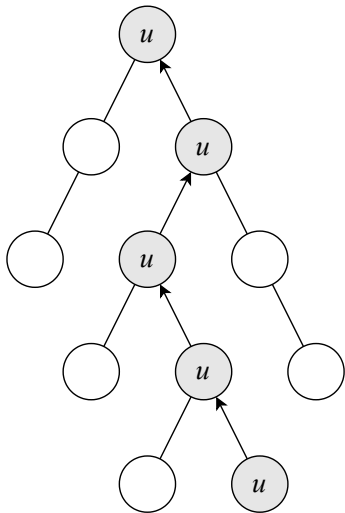


If leaf node is reached $\rightarrow$ **expand** the search tree by adding a new node for the reached state $\hat{s}'$

The new node is initialized with

- $N(\hat{s}^I, \hat{a}) = 0$ for all actions $\hat{a}$
- An initial value estimate $Q(\hat{s}^I, \hat{a})$ for all actions $\hat{a}$, e.g. from a learned value function, heuristic, or random samples of outcomes.

Monte Carlo Tree Search – Backpropagation



Once a value estimate u for the leaf node is obtained $\rightarrow$ **backpropagate** rewards and value estimates starting from the leaf node up to the root node.

For each visited state-action pair $(\hat{s}^\tau, \hat{a}^\tau)$, we increment the visitation count and update the value:

$$Q(\hat{s}^\tau, \hat{a}^\tau) \leftarrow Q(\hat{s}^\tau, \hat{a}^\tau) + \frac{1}{N(\hat{s}^\tau, \hat{a}^\tau)} [u - Q(\hat{s}^\tau, \hat{a}^\tau)]$$

(Any RL TD update rule can be used to update the value estimates.)

Monte Carlo Tree Search – Action Selection

Simulation, expansion, and backpropagation steps are repeated at every time step to iteratively grow the search tree and obtain better value estimates.

Following k simulations from the current state s^t , the best action is selected. This process can be done by choosing the action with:

- highest value estimate: $BestAction(s^t) = \arg \max_{\hat{a} \in A} Q(s^t, \hat{a})$
- highest visitation count: $BestAction(s^t) = \arg \max_{\hat{a} \in A} N(s^t, \hat{a})$

Monte Carlo Tree Search – Pseudocode

Algorithm Monte Carlo tree search (MCTS) for MDPs

```
1: Repeat for every episode:
2:   for  $t = 0, 1, 2, 3, \dots$  do
3:     Observe current state  $s^t$ 
4:     for  $k$  simulations do
5:        $\tau \leftarrow t$ 
6:        $\hat{s}^\tau \leftarrow s^t$  ▷ Perform simulation
7:       while  $\hat{s}^\tau$  is non-terminal and  $\hat{s}^\tau$ -node exists in tree do
8:          $\hat{a}^\tau \leftarrow \text{ExploreAction}(\hat{s}^\tau)$ 
9:          $\hat{s}^{\tau+1} \sim \mathcal{T}(\cdot \mid \hat{s}^\tau, \hat{a}^\tau)$ 
10:         $\tilde{r}^\tau \leftarrow \mathcal{R}(\hat{s}^\tau, \hat{a}^\tau, \hat{s}^{\tau+1})$ 
11:         $\tau \leftarrow \tau + 1$ 
12:      if  $\hat{s}^\tau$ -node does not exist in tree then
13:         $\text{InitializeNode}(\hat{s}^\tau)$  ▷ Expand tree
14:      while  $\tau > t$  do ▷ Backpropagate
15:         $\tau \leftarrow \tau - 1$ 
16:         $\text{Update}(Q, \hat{s}^\tau, \hat{a}^\tau)$ 
17:      Select action  $a^t$  for state  $s^t$ :
18:       $\pi^t \leftarrow \text{BestAction}(s^t)$ 
19:       $a^t \sim \pi^t$ 
```

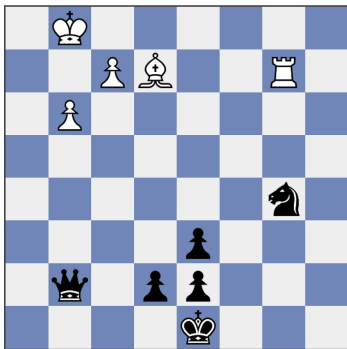
Note

MCTS assumes known transition function $\mathcal{T}$ and reward function $\mathcal{R}$.

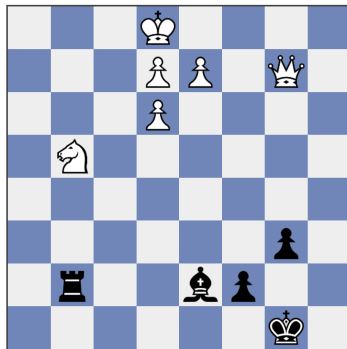
Can learn estimates of these functions from data to simulate possible outcomes of the game.

Self-Play Monte Carlo Tree Search

In zero-sum games with symmetrical roles and egocentric observations, agents can use the same policy to control both players → learn a policy in **self-play**



(a) Agent 1 perspective



(b) Agent 2 perspective

AlphaZero – Self-Play MCTS with Deep Learning

AlphaZero: combine self-play MCTS with deep learning to learn value estimates and policies → reached superhuman performance in Go, chess, and shogi!

Learn functions with parameters θ conditioned on state s :

- Value estimate $V(s; \theta)$
- Policy $\pi(\cdot | s; \theta)$

For each episode, a triplet (s, π, z) of data is stored where

- s are the states
- π are policy distributions computed by *BestAction*
- z is the game outcome (+1 for win, -1 for loss, 0 for draw)



AlphaZero – Self-Play MCTS with Deep Learning

The network is randomly initialized and trained using sampled batches of data to minimise the following combined loss:

$$\begin{aligned}\mathcal{L}(\theta) &= \mathcal{L}_{\text{value}} + \mathcal{L}_{\text{policy}} + c \|\theta\|^2 \\ \mathcal{L}_{\text{value}} &= \mathbb{E}_{(s, \pi, z) \sim \mathcal{D}} \left[(V(s; \theta) - u)^2 \right] \\ \mathcal{L}_{\text{policy}} &= \mathbb{E}_{(s, \pi, z) \sim \mathcal{D}} \left[\pi^\top \log \pi(\cdot \mid s; \theta) \right]\end{aligned}$$

For exploration, AlphaZero combines a UCB policy with the learned policy:

$$\hat{a}^\tau = \begin{cases} \hat{a} & \text{if } N(\hat{s}^\tau, \hat{a}) = 0 \\ \arg \max_{\hat{a} \in A} \left(Q(\hat{s}^\tau, \hat{a}) + C(\hat{s}^\tau) P(\hat{s}^\tau, \hat{a}) \frac{\sqrt{N(\hat{s}^\tau)}}{1 + N(\hat{s}^\tau, \hat{a})} \right) & \text{otherwise} \end{cases}$$

with the additional exploration rate $C(\hat{s}^\tau)$.

Population-Based Training

Population-Based Training – Self-Play for General-Sum Games

Problem

With MCTS, we focused on policy self-play in **two-agent zero-sum** games. Can we extend the idea of self-play to **general-sum** games with **more than two agents**?

Population-based training is a generalisation of self-play to general-sum games:

- Maintain a **population of policies** representing possible strategies of the agent
- Evolve populations so they become more effective against the populations of other agents
- We denote the population of policies for agent i at generation k as Π_i^k .

Population-Based Training – Overview

First, initialize a population of policies for each agent (e.g. random policies).

For each generation, population-based training then follows two steps:

- **Evaluation:** evaluate the performance of the policies in the population by playing games against policies of the current populations of all other agents
- **Evolution:** based on evaluation results, evolve the populations of all agents. This can be done by selecting a subset of high-performing policies, mutating existing policies, or adding new policies to the population.

This process is repeated until convergence or for a fixed number of generations.

Policy Space Response Oracles (PSRO)

Policy space response oracles (PSRO) is a population-based MARL algorithm with the following steps:

1. Initialize populations of random policies for each agent
2. Construct a **meta-game** M^k at generation k from the populations of all agents as a (non-repeated) normal-form game with
 - Actions: policies of agent populations, i.e. $A_i = \Pi_i^k$
 - Rewards: returns of joint policies $\langle \pi_1, \dots, \pi_n \rangle$, i.e. $\mathcal{R}_i(\pi_1, \dots, \pi_n) = U_i(\pi_1, \dots, \pi_n)$.

M^k	$\pi_2^{(1)}$	$\pi_2^{(2)}$	$\dots$	$\pi_2^{(k)}$
$\pi_1^{(1)}$	0, 1	1, 2	$\dots$	0, 3
$\pi_1^{(2)}$	2, 1	0, 1	$\dots$	1, 1
$\vdots$	$\vdots$	$\vdots$	$\cdot$	$\vdots$
$\pi_1^{(k)}$	5, 1	0, 1	$\dots$	4, 3

Policy Space Response Oracles – Construct Meta-Game

M^k	$\pi_2^{(1)}$	$\pi_2^{(2)}$	$\dots$	$\pi_2^{(k)}$
$\pi_1^{(1)}$	0, 1	1, 2	$\dots$	0, 3
$\pi_1^{(2)}$	2, 1	0, 1	$\dots$	1, 1
$\vdots$	$\vdots$	$\vdots$	$\cdot$	$\vdots$
$\pi_1^{(k)}$	5, 1	0, 1	$\dots$	4, 3

Problem

Need to compute the expected returns of each agent for any joint policy $\langle \pi_1, \dots, \pi_n \rangle$ in the meta-game. How can we do this?

Solution

Compute average returns of each agent over multiple episodes of the underlying game with respective joint policy $\rightarrow$ converges to expected returns in the limit.

Policy Space Response Oracles – Solve Meta-Game

δ_1^k/δ_2^k					
	M^k	$\pi_2^{(1)}$	$\pi_2^{(2)}$	$\dots$	$\pi_2^{(k)}$
	$\pi_1^{(1)}$	0, 1	1, 2	$\dots$	0, 3
	$\pi_1^{(2)}$	2, 1	0, 1	$\dots$	1, 1
	$\vdots$	$\vdots$	$\vdots$	$\cdot$	$\vdots$
	$\pi_1^{(k)}$	5, 1	0, 1	$\dots$	4, 3

Compute a **solution to the meta-game** M^k following some solution concept, e.g. a Nash equilibrium.

Solution to the meta-game: distributions δ_i^k over policies in the population of agent i (for each agent $i \in I$).

Policy Space Response Oracles – Add Best-Response Policies

M^k	$\pi_2^{(1)}$	$\pi_2^{(2)}$	$\dots$	$\pi_2^{(k)}$	π'_2
$\pi_1^{(1)}$	0, 1	1, 2	$\dots$	0, 3	?
$\pi_1^{(2)}$	2, 1	0, 1	$\dots$	1, 1	?
$\vdots$	$\vdots$	$\vdots$	$\cdot$	$\vdots$	?
$\pi_1^{(k)}$	5, 1	0, 1	$\dots$	4, 3	?
π'_1	?	?	?	?	?

Each agent i determines an effective **oracle policy** π'_i against the solution distribution of the other agents and adds this policy to its population:

$$\Pi_i^{k+1} = \Pi_i^k \cup \{\pi'_i\}$$

For example, agent i might determine its best-response policy

$$\pi'_i \in \arg \max_{\pi_i} \mathbb{E}_{\pi_{-i} \sim \delta_{-i}^k} [U_i(\langle \pi_i, \pi_{-i} \rangle)]$$

by training a policy π'_i using RL against sampled policies of the other agents.

Policy Space Response Oracles – Pseudocode

Algorithm Policy space response oracles (PSRO)

- 1: Initialize populations Π_i^1 for all $i \in I$ (e.g., random policies)
 - 2: **for** each generation $k = 1, 2, 3, \dots$ **do**
 - 3: Construct meta-game M^k from current populations $\{\Pi_i^k\}_{i \in I}$
 - 4: Use meta-solver on M^k to obtain distributions $\{\delta_i^k\}_{i \in I}$
 - 5: **for** each agent $i \in I$ **do** ▷ Train best-response policies
 - 6: **for** each episode $e = 1, 2, 3, \dots$ **do**
 - 7: Sample policies for other agents $\pi_{-i} \sim \delta_{-i}^k$
 - 8: Use single-agent RL to train π'_i wrt. π_{-i} in underlying game G
 - 9: Grow population $\Pi_i^{k+1} \leftarrow \Pi_i^k \cup \{\pi'_i\}$
-

Repeat this process until policy populations converge (new best-response policies are already in the respective populations), or for a fixed number of generations.

Policy Space Response Oracles in Rock-Paper-Scissors

If PSRO computes exact Nash equilibria solutions to the meta-game, and computes exact best-response policies, then the population distributions of PSRO converge to the Nash equilibrium of the underlying game.

For example for rock-paper-scissors for two agents, with initial populations deterministically choosing rock and paper, respectively:

k	Π_1^k	Π_2^k	δ_1^k	δ_2^k	π'_1	π'_2
1	<u>R</u>	<u>P</u>	1	1	S	P
2	R, <u>S</u>	P	(0, 1)	1	S	R
3	R,S	<u>R</u> ,P	$(\frac{2}{3}, \frac{1}{3})$	$(\frac{2}{3}, \frac{1}{3})$	P	R/P
4	R, <u>P</u> ,S	R,P	$(0, \frac{2}{3}, \frac{1}{3})$	$(\frac{1}{3}, \frac{2}{3})$	R	S
5	R,P,S	R,P, <u>S</u>	$(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$	$(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$	R/P/S	R/P/S

AlphaStar – GrandMaster in StarCraft II

StarCraft II is a real-time strategy game for two or more players in which players have to collect resources, build infrastructure and armies to defeat their opponents.



Figure: StarCraft II game, image source: <https://deepmind.google/discover/blog/alphastar-mastering-the-real-time-strategy-game-starcraft-ii/>.

AlphaStar – GrandMaster in StarCraft II

AlphaStar pre-trains policies for each race using supervised imitation learning on human demonstrations from observation histories and game statistics.

After pre-training, AlphaStar trains its policies with deep RL using a population-based training approach called **league training**: for each race, maintain a league with three types of populations, each with a different objective:

- **Main agents**: largely trained in self-play and against any prior policy
- **Exploiter agents**: largely trained against the current main agents to learn to exploit their weaknesses
- **League exploiter agents**: trained against all agents in the league to identify effective strategies missing in the league

Agents of each type are added to the league whenever they become effective (measured by win rates) against their respective opponents.

AlphaStar – GrandMaster in StarCraft II

During population-based training, AlphaStar computes distributions over the policies in the league to train any policy π'_i against using **prioritized fictitious self-play** (PFSP):

$$\delta_i^k(\pi_i) \propto f(\Pr[\pi'_i \text{ wins against } \pi_i])$$

which is proportional to the probability of the agent winning against the policy π_i .

and $f: [0, 1] \rightarrow [0, \infty)$ is a weighting function with two components:

- f_{hard} : focus on the most difficult opponent policies for π'_i
- f_{var} focus on opponent policies that are at a similar level of performance as π'_i

Win probabilities are estimated by running multiple matches of π'_i versus π_i .

→ reached “GrandMaster” level in StarCraft II (top 0.2% of ranked human players).